



المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT



Projet de Fin d'Année

Filière : Sécurité IT et Confiance Numérique

Détection d'intrusions réseau par apprentissage automatique à l'aide de jeux de données publics

Présenté par

Ait El Hadj Mohamed

Année Universitaire 2025- 2026

DÉDICACES

Je dédie ce modeste travail :

À Dieu Tout-Puissant, qui m'a accordé la santé, la force, le courage et la persévérance nécessaires pour mener à bien ce parcours.

À ma chère mère, pour son amour inconditionnel, son soutien indéfectible, ses sacrifices, ses encouragements et ses précieux conseils. Qu'elle trouve à travers ce travail l'expression de ma profonde gratitude et de mon immense reconnaissance.

À mon cher père, pour tous les sacrifices qu'il a consentis afin de m'offrir les meilleures conditions de réussite. Merci pour les valeurs, l'éducation et le soutien constant qui m'ont permis d'avancer avec confiance tout au long de mon parcours.

À mes sœurs, pour leur affection, leurs encouragements permanents et leur soutien moral.

À mes amis, pour leur présence, leur confiance, leur soutien et tous les moments de partage qui ont rendu ce parcours plus agréable.

À mes enseignants, pour la qualité de leur enseignement, leur disponibilité, leurs conseils avisés et leur accompagnement tout au long de ma formation.

Enfin, à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail, je tiens à exprimer ma sincère reconnaissance.

***« Veuillez trouver dans ce travail le fruit de vos bénédictions, et l'expression de ma
profonde gratitude. »***

Ait El Hadj Mohamed

REMERCIEMENTS

Au terme de ce travail, je tiens tout d'abord à exprimer ma profonde gratitude à **Allah, le Tout-Puissant**, pour m'avoir accordé la santé, la force, la patience et la persévérance nécessaires à la réalisation de ce projet.

Je tiens à adresser mes sincères remerciements à toutes les personnes qui, par leur soutien, leurs conseils et leurs encouragements, ont contribué à la réussite de ce stage et à l'aboutissement de ce travail.

J'exprime ma reconnaissance à **Monsieur le Directeur de l'École Nationale Supérieure de l'Intelligence Artificielle et des Sciences des Données (ENSIASD), Pr. Jilali ANTARI**, pour son engagement, sa vision et les efforts qu'il déploie afin d'offrir un environnement de formation de qualité.

Je tiens à remercier chaleureusement **mon encadrant pédagogique, Pr. Mohamed Saad AZIZI**, pour son accompagnement, sa disponibilité, ses conseils avisés et son soutien tout au long de ce projet. Son expertise et ses orientations ont été d'une grande importance dans la réalisation de ce travail.

Mes remerciements les plus sincères s'adressent à **Monsieur SAAD BELLAOUI**, mon encadrant de stage au sein de **SOREMED**, pour son accueil, sa confiance, sa disponibilité, ses précieux conseils ainsi que son accompagnement tout au long de cette expérience professionnelle.

Je souhaite également exprimer toute ma gratitude à **l'ensemble du personnel de SOREMED** pour leur accueil chaleureux, leur disponibilité et l'excellente ambiance de travail qui m'ont permis d'effectuer ce stage dans les meilleures conditions.

Enfin, j'adresse mes sincères remerciements aux **membres du jury**, qui ont accepté d'évaluer ce travail. Je leur suis reconnaissant pour le temps qu'ils y consacrent ainsi que pour leurs remarques et recommandations, qui contribueront à l'amélioration de ce projet.

RÉSUMÉ

Dans le cadre de mon stage effectué au sein de **SOREMED**, ce travail porte sur la conception et le développement d'un système de détection d'intrusions (IDS) basé sur des techniques d'apprentissage automatique afin d'améliorer la sécurité des réseaux informatiques. Face à l'évolution constante des cyberattaques, les méthodes traditionnelles de détection montrent certaines limites, ce qui rend nécessaire l'utilisation d'approches intelligentes capables d'identifier automatiquement les activités malveillantes.

Le projet s'appuie sur le jeu de données public **CICIDS2017**, qui contient un trafic réseau réaliste composé de flux normaux et de différentes catégories d'attaques. Une phase de prétraitement des données a été réalisée afin d'assurer leur qualité, incluant le nettoyage, la suppression des doublons, la sélection des attributs pertinents ainsi que la préparation des données pour l'entraînement des modèles.

Deux approches de classification ont été étudiées : la **classification binaire**, permettant de distinguer le trafic normal des attaques, et la **classification multi classe**, visant à identifier précisément le type d'attaque détecté. Plusieurs algorithmes de Machine Learning ont été évalués, notamment **Decision Tree**, **Random Forest**, **XGBoost** et un réseau de neurones **MLP (Multi-Layer Perceptron)**.

Les performances des modèles ont été comparées à l'aide de plusieurs métriques telles que l'exactitude (*Accuracy*), la précision (*Precision*), le rappel (*Recall*), le score **F1** pondéré et le **Macro F1-score**, ainsi qu'au moyen d'une validation croisée (*K-Fold Cross Validation*) afin d'évaluer leur capacité de généralisation. Les résultats obtenus montrent que le modèle **Decision Tree** offre un excellent compromis entre performances, rapidité d'exécution et simplicité d'interprétation, ce qui en fait le modèle retenu pour le déploiement.

Enfin, une application web interactive développée avec **Streamlit** a été mise en place afin de permettre la prédiction en temps réel à partir des caractéristiques d'un flux réseau ou d'un fichier CSV, ainsi que la visualisation des résultats et des performances du modèle.

Mots-clés : Cybersécurité, Détection d'intrusions, IDS, Machine Learning, Apprentissage automatique, CICIDS2017, Decision Tree, Random Forest, XGBoost, MLP, Classification binaire, Classification multi classe, Streamlit.

ABSTRACT

As part of my internship at **SOREMED**, this project focuses on the design and development of a **Machine Learning-based Intrusion Detection System (IDS)** to improve network security. With the continuous increase in the number and sophistication of cyberattacks, traditional detection methods are no longer sufficient to effectively identify malicious activities. Therefore, intelligent approaches based on Machine Learning have become a promising solution for detecting network intrusions.

The proposed system is built using the **CICIDS2017** public dataset, which contains realistic network traffic including both normal activities and multiple categories of cyberattacks. A comprehensive data preprocessing phase was performed, including data cleaning, duplicate removal, feature selection, and data preparation to ensure high-quality inputs for model training.

Two classification approaches were investigated: **binary classification**, which distinguishes normal traffic from malicious traffic, and **multiclass classification**, which identifies the exact type of detected attack. Several Machine Learning algorithms were evaluated, including **Decision Tree**, **Random Forest**, **XGBoost**, and a **Multi-Layer Perceptron (MLP)** neural network.

The models were assessed using standard performance metrics such as **Accuracy**, **Precision**, **Recall**, **Weighted F1-score**, and **Macro F1-score**. In addition, **K-Fold Cross Validation** was performed to evaluate the robustness and generalization capability of the selected models. The experimental results demonstrated that the **Decision Tree** algorithm provides the best balance between predictive performance, computational efficiency, and interpretability, making it the most suitable model for deployment.

Finally, an interactive web application was developed using **Streamlit** to allow real-time intrusion detection from manually entered network features or uploaded CSV files.

Keywords: Cybersecurity, Intrusion Detection System (IDS), Machine Learning, CICIDS2017, Decision Tree, Random Forest, XGBoost, Multi-Layer Perceptron (MLP), Binary Classification, Multiclass Classification, Streamlit.

Table des matières

DÉDICACES	3
REMERCIEMENTS.....	4
RÉSUMÉ.....	5
ABSTRACT	6
Chapitre 1 : Environnement du stage.....	13
Introduction	13
1. Présentation de SOREMED	13
1.1 Historique.....	13
1.2 Missions et activités	14
1.3 Organisation de l'entreprise	14
1.4 Organigramme de SOREMED	15
2. Contexte du stage.....	16
2.1 Présentation du projet.....	16
2.2 Problématique.....	17
2.3 Objectifs du projet.....	17
2.4 Cahier des charges.....	17
3. Planification du projet	18
3.1 Méthodologie adoptée	18
Conclusion.....	19
Chapitre 2 : État de l'art.....	20
Introduction	20
1. La cybersécurité et les systèmes de détection d'intrusions	20
1.1 La cybersécurité	20
1.2 Les cyberattaques	20
1.3 Les systèmes de détection d'intrusions (IDS).....	21
1.4 Les types d'IDS	22
1.5 Les limites des IDS traditionnels.....	22
2. L'apprentissage automatique appliqué à la détection d'intrusions	23
2.1 Introduction au Machine Learning.....	23
2.2 Les types d'apprentissage	23
2.3 La classification binaire	24
2.4 La classification multi classe.....	24
3. Les algorithmes de Machine Learning	24
3.1 Decision Tree	24

3.2 Random Forest.....	25
3.3 XGBoost	25
3.4 Multi-Layer Perceptron (MLP).....	26
4. Jeu de données et évaluation des modèles	26
4.1 Présentation du jeu de données CICIDS2017	26
4.2 Prétraitement des données.....	27
4.3 Métriques d'évaluation	27
4.4 Validation croisée (K-Fold Cross Validation).....	29
Conclusion.....	29
Chapitre 3 : Conception et modélisation	31
Introduction	31
1. Analyse des besoins.....	31
1.1 Besoins fonctionnels	31
1.2 Besoins non fonctionnels	31
2. Architecture générale du système	32
2.1 Architecture proposée	32
2.2 Description des composants.....	32
3. Conception de la solution de Machine Learning	33
3.1 Pipelines de traitement des données	33
3.2 Prétraitement des données.....	34
3.3 Sélection des caractéristiques	35
3.4 Processus d'entraînement des modèles	35
4. Conception de l'application web	35
4.1 Architecture de l'application Streamlit.....	35
4.2 Description des principales interfaces	36
Conclusion.....	36
Chapitre 4 : Expérimentation et résultats.....	37
Introduction	37
1. Environnement de développement.....	37
1.1 Langage de programmation	37
1.2 Bibliothèques et frameworks	37
1.3 Outils de développement.....	38
1.4 Jeu de données utilisée	38
2. Réalisation du système	39
2.1 Présentation du jeu de données	39
2.2 Analyse exploratoire des données (EDA).....	41

2.3 Prétraitement des données.....	42
2.4 Classification binaire.....	44
2.5 Classification multi classe	45
2.6 Développement de l'application web	45
3. Résultats expérimentaux.....	45
3.1 Résultats de la classification binaire	46
3.2 Résultats de la classification multi classe	48
3.3 Comparaison des modèles de Machine Learning	51
3.4 Validation par K-Fold Cross Validation	52
3.5 Analyse et interprétation des résultats.....	54
4. Présentation de l'application	54
4.1 Page d'accueil.....	54
4.2 Tableau de bord	54
4.3 Interface de prédiction	54
4.4 Affichage des résultats.....	54
Conclusion.....	54

LISTE DES ABRÉVIATIONS

Abréviation	Signification
IA	Intelligence Artificielle
ML	Machine Learning (Apprentissage automatique)
DL	Deep Learning (Apprentissage profond)
IDS	Intrusion Detection System (Système de détection d'intrusions)
IPS	Intrusion Prevention System (Système de prévention des intrusions)
DoS	Denial of Service (Déni de service)
DDoS	Distributed Denial of Service (Déni de service distribué)
MLP	Multi-Layer Perceptron (Perceptron multicouche)
CSV	Comma-Separated Values (Valeurs séparées par des virgules)
API	Application Programming Interface (Interface de programmation d'applications)
CICIDS2017	Canadian Institute for Cybersecurity Intrusion Detection System 2017
K-Fold	Validation croisée en K plis
F1-score	Moyenne harmonique de la précision et du rappel
TP	True Positive (Vrai positif)
TN	True Negative (Vrai négatif)
FP	False Positive (Faux positif)
FN	False Negative (Faux négatif)
ENSIASD	École Nationale Supérieure de l'Intelligence Artificielle et des Sciences des Données
SOREMED	Société de Répartition des Médicaments

LISTE DES FIGURES

Figure 1: Organigramme de SOREMED.....	16
Figure 2: Principe de fonctionnement de la validation croisée K-Fold ($K = 5$)	29
Figure 3: Architecture générale du système de détection d'intrusions.....	32
Figure 4: Pipeline complet du projet	34
Figure 5: Architecture générale de l'application.....	36
Figure 6: Python Logo.....	37
Figure 7: Répartition des classes dans les différents fichiers du jeu de données.....	42
Figure 8: Pipeline de nettoyage des données.....	43
Figure 9: Matrices de confusion des algorithmes de ML avec destination port.....	46
Figure 10: Matrices de confusion des modèles sans Destination Port.....	47

INTRODUCTION GÉNÉRALE

Avec la transformation numérique des entreprises et la croissance constante des échanges de données sur les réseaux informatiques, les cyberattaques sont devenues de plus en plus fréquentes, sophistiquées et difficiles à détecter. Les organisations sont aujourd'hui confrontées à de nombreuses menaces, telles que les attaques par déni de service, les intrusions, les scans de ports ou encore les attaques par force brute, pouvant compromettre la confidentialité, l'intégrité et la disponibilité des systèmes d'information. Dans ce contexte, la mise en place de mécanismes efficaces de détection d'intrusions constitue un enjeu majeur en matière de cybersécurité.

Les systèmes de détection d'intrusions (IDS) jouent un rôle essentiel dans la surveillance du trafic réseau et l'identification des comportements malveillants. Cependant, les approches traditionnelles reposant sur des signatures présentent des limites face à l'apparition de nouvelles formes d'attaques. L'intégration des techniques d'apprentissage automatique (Machine Learning) permet de concevoir des systèmes plus intelligents, capables d'apprendre à partir des données et de détecter automatiquement des comportements anormaux.

C'est dans ce cadre que s'inscrit ce projet de stage, réalisé au sein de **SOREMED**, dont l'objectif est de concevoir et de développer un système de détection d'intrusions basé sur le Machine Learning. Le projet s'appuie sur le jeu de données public **CICIDS2017**, qui fournit un trafic réseau représentatif contenant à la fois des communications normales et plusieurs catégories d'attaques. Après une phase de prétraitement des données, plusieurs modèles de classification ont été entraînés, évalués et comparés afin d'identifier la solution la plus performante. Une application web interactive développée avec **Streamlit** a ensuite été mise en œuvre pour permettre la prédiction et la visualisation des résultats.

Le présent rapport est organisé en quatre chapitres.

- Le **premier chapitre** présente l'organisme d'accueil, **SOREMED**, ainsi que le contexte du stage.
- Le **deuxième chapitre** expose les notions théoriques relatives à la cybersécurité, aux systèmes de détection d'intrusions et au Machine Learning.
- Le **troisième chapitre** décrit la conception et la réalisation du système proposé.
- Enfin, le **quatrième chapitre** présente les résultats expérimentaux obtenus, leur analyse ainsi que l'application web développée.

Chapitre 1 : Environnement du stage

Introduction

L'industrie pharmaceutique marocaine occupe une place stratégique dans l'économie nationale et joue un rôle essentiel dans le système de santé. Depuis son développement dans les années 1960, ce secteur a connu une évolution remarquable grâce à l'acquisition d'un savoir-faire industriel et à l'intégration de nouvelles technologies. Aujourd'hui, il contribue à assurer la disponibilité des médicaments sur l'ensemble du territoire national tout en répondant aux exigences de qualité, de sécurité et de traçabilité.

La distribution des produits pharmaceutiques constitue un maillon fondamental de cette industrie. Elle est encadrée par une réglementation rigoureuse garantissant la conservation, le transport et la mise à disposition des médicaments dans des conditions conformes aux normes en vigueur. Cette chaîne logistique implique les laboratoires pharmaceutiques, les grossistes répartiteurs et les pharmacies d'officine afin d'assurer un approvisionnement continu des établissements de santé.

C'est dans ce contexte que s'inscrit **SOREMED (Société Régionale de Répartition des Médicaments)**, entreprise spécialisée dans la répartition et la distribution de produits pharmaceutiques au Maroc. Ce chapitre présente l'entreprise d'accueil, son organisation ainsi que le contexte dans lequel s'est déroulé le stage.

1. Présentation de SOREMED

1.1 Historique

Créée le **1er février 2001**, **SOREMED (Société Régionale de Répartition des Médicaments)** est une société anonyme (SA) de droit marocain dont le siège social est situé dans la **Zone Industrielle Tassila à Agadir**. Depuis sa création, l'entreprise s'est spécialisée dans la répartition et la distribution des produits pharmaceutiques sur l'ensemble du territoire marocain.

Grâce à son expérience, à son réseau logistique et au respect des normes réglementaires, SOREMED est devenue un acteur reconnu dans la chaîne de distribution pharmaceutique. Son développement repose sur la qualité de ses services, la fiabilité de son système de distribution

et son engagement à garantir la disponibilité des médicaments auprès des professionnels de santé.

1.2 Missions et activités

La mission principale de SOREMED est d'assurer la répartition et la distribution des médicaments et des produits parapharmaceutiques auprès des pharmacies d'officine dans les meilleures conditions de qualité, de sécurité et de traçabilité.

Pour atteindre cet objectif, l'entreprise assure plusieurs activités, notamment :

- ✓ La réception et le stockage des produits pharmaceutiques conformément aux normes en vigueur
- ✓ La gestion des stocks et le contrôle de la traçabilité des médicaments
- ✓ La préparation et l'expédition des commandes destinées aux pharmacies
- ✓ La distribution des produits sur l'ensemble du territoire marocain
- ✓ Le respect des exigences réglementaires relatives au transport et à la conservation des médicaments.

Par son activité, SOREMED contribue activement à garantir un approvisionnement continu des établissements pharmaceutiques et participe ainsi au bon fonctionnement du système de santé marocain.

1.3 Organisation de l'entreprise

SOREMED adopte une organisation structurée permettant d'assurer une gestion efficace de ses activités administratives, commerciales, logistiques et techniques. L'entreprise est dirigée par **Monsieur Hamid OUAHBI**, Directeur Général, et s'appuie sur plusieurs services complémentaires qui collaborent afin d'assurer le bon déroulement de la chaîne de distribution des produits pharmaceutiques.

Les principaux services de l'entreprise comprennent notamment :

- ✓ La Direction Générale
- ✓ Le service Administratif et Financier
- ✓ Le service Commercial
- ✓ Le service Logistique et Distribution

- ✓ Le service Qualité
- ✓ Le service Informatique.

Cette organisation permet d'assurer une coordination efficace entre les différents services tout en garantissant le respect des normes de qualité imposées dans le secteur pharmaceutique.

Le tableau suivant présente les principales informations relatives à SOREMED.

Désignation	Information
Raison sociale	SOREMED (Société Régionale de Répartition des Médicaments)
Forme juridique	Société Anonyme (SA)
Activité	Répartition et distribution de produits pharmaceutiques
Date de création	1er février 2001
Siège social	Zone Industrielle Tassila, N°176, Agadir
Directeur Général	M. Hamid OUAHBI
Capital social	50 000 000 MAD
Registre de commerce	1366
Patente	49760929
Téléphone	05 28 33 22 66
Fax	05 28 33 21 98
Superficie	5 000 m ²
Champ d'action	Ensemble du territoire marocain

1.4 Organigramme de SOREMED

L'organisation interne de SOREMED repose sur une structure hiérarchique permettant d'assurer une coordination efficace entre les différents services. L'organigramme ci-dessous présente les principales directions et les relations fonctionnelles qui existent entre elles.

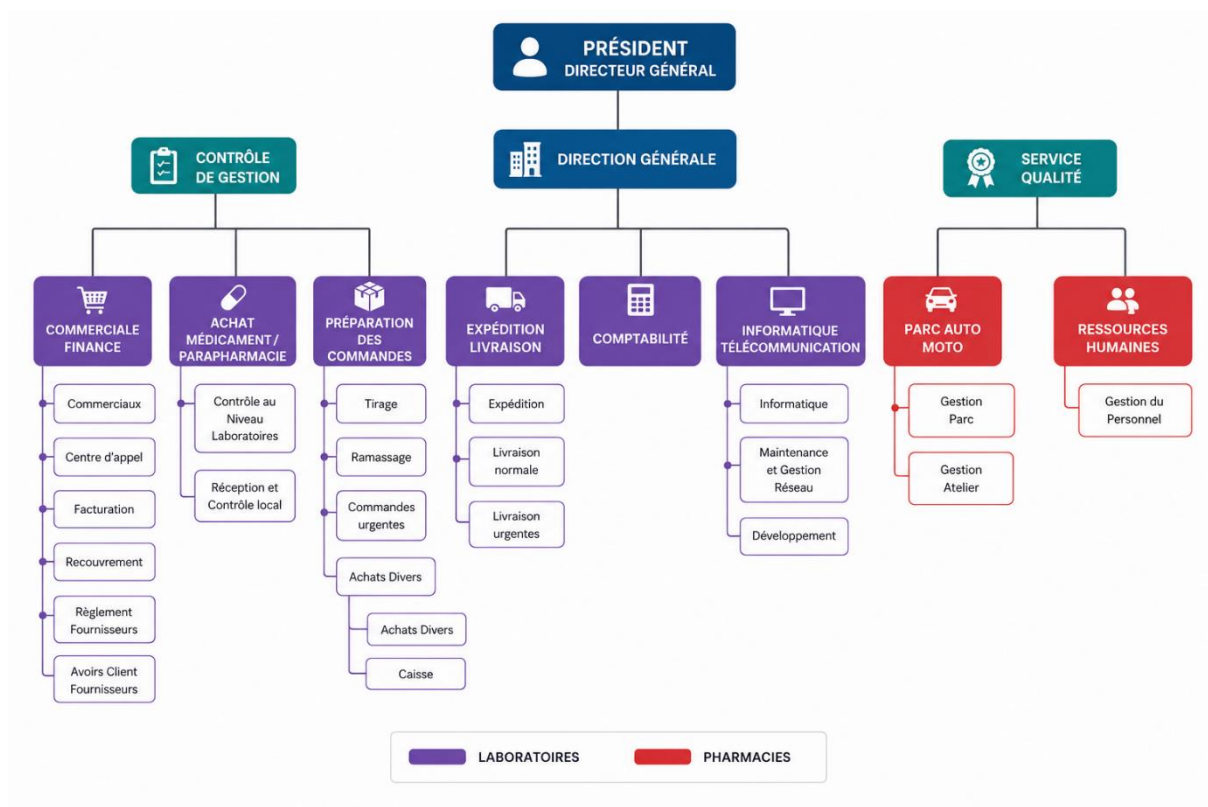


Figure 1: Organigramme de SOREMED

2. Contexte du stage

Face à l'évolution constante des cybermenaces, les entreprises sont amenées à renforcer la sécurité de leurs systèmes d'information afin de protéger leurs données et leurs infrastructures. Dans ce contexte, SOREMED souhaite explorer des solutions intelligentes capables d'améliorer la détection des activités malveillantes sur les réseaux informatiques.

C'est dans cette perspective que s'inscrit ce projet de stage, dont l'objectif est de concevoir un système de détection d'intrusions basé sur les techniques de Machine Learning. Le projet couvre l'ensemble des étapes nécessaires, depuis le prétraitement des données jusqu'au développement d'une application web permettant la prédiction des attaques.

2.1 Présentation du projet

Le projet réalisé consiste à développer un système de détection d'intrusions exploitant des algorithmes d'apprentissage automatique afin d'identifier les activités malveillantes présentes dans le trafic réseau.

Pour cela, le jeu de données public **CICIDS2017** a été utilisé pour entraîner et évaluer plusieurs modèles de classification, notamment **Decision Tree**, **Random Forest**, **XGBoost** et **Multi-Layer Perceptron (MLP)**. Une étude comparative a ensuite été menée afin de

sélectionner le modèle le plus performant. Enfin, une application web interactive développée avec **Streamlit** a été réalisée pour permettre la prédiction en temps réel des intrusions à partir des caractéristiques d'un flux réseau ou d'un fichier CSV.

2.2 Problématique

Les systèmes traditionnels de détection d'intrusions reposent principalement sur des bases de signatures, ce qui limite leur capacité à détecter les attaques nouvelles ou inconnues. Par ailleurs, l'augmentation du volume et de la complexité du trafic réseau rend leur efficacité de plus en plus limitée.

La problématique de ce projet est donc la suivante :

Comment concevoir un système de détection d'intrusions performant, basé sur le Machine Learning, capable de détecter efficacement les attaques réseau et de les classifier avec un bon niveau de précision ?

2.3 Objectifs du projet

L'objectif principal de ce stage est de concevoir et développer un système intelligent de détection d'intrusions basé sur les techniques de Machine Learning.

Pour atteindre cet objectif, plusieurs objectifs spécifiques ont été définis :

- Étudier les systèmes de détection d'intrusions et les techniques de Machine Learning
- Prétraiter le jeu de données CICIDS2017 afin d'obtenir des données exploitables
- Entraîner et évaluer plusieurs modèles de classification
- Comparer les performances des modèles à l'aide de différentes métriques d'évaluation
- Sélectionner le modèle le plus performant pour le déploiement
- Développer une application web interactive permettant la détection et la classification des intrusions.

2.4 Cahier des charges

Le cahier des charges définit l'ensemble des exigences que doit satisfaire le système développé afin de répondre aux besoins exprimés dans le cadre de ce projet. Ces exigences sont réparties en deux catégories : les exigences fonctionnelles et les exigences non fonctionnelles.

2.4.1 Exigences fonctionnelles

Le système développé doit permettre de :

- Détecter automatiquement les intrusions à partir des données du trafic réseau
- Identifier le type d'attaque détectée dans le cadre de la classification multiclasse
- Effectuer des prédictions à partir d'une saisie manuelle des caractéristiques réseau
- Importer un fichier CSV afin de réaliser des prédictions sur plusieurs observations
- Afficher les résultats de manière claire et compréhensible à travers une interface web interactive
- Comparer les performances des différents modèles de Machine Learning.

2.4.2 Exigences non fonctionnelles

Le système doit également respecter plusieurs contraintes de qualité, notamment :

- Assurer une précision élevée dans la détection et la classification des intrusions
- Fournir des temps de réponse rapides pour les prédictions
- Offrir une interface utilisateur simple, intuitive et ergonomique
- Garantir la fiabilité et la stabilité de l'application
- Faciliter la maintenance et l'évolution du système
- Assurer la sécurité des données traitées par l'application.

3. Planification du projet

La réalisation de ce projet a été organisée selon une démarche progressive permettant de structurer les différentes étapes de développement. Cette planification a facilité le suivi de l'avancement du projet tout en assurant une répartition cohérente des tâches, depuis l'étude préliminaire jusqu'au développement de l'application web.

3.1 Méthodologie adoptée

Afin d'atteindre les objectifs fixés, une méthodologie de travail structurée a été adoptée. Le projet s'est déroulé selon les principales phases suivantes :

- Étude bibliographique et analyse des besoins ;

- Collecte et préparation du jeu de données CICIDS2017 ;
- Prétraitement et analyse des données ;
- Entraînement et évaluation des modèles de Machine Learning ;
- Validation des performances des modèles ;
- Développement de l'application web avec Streamlit ;
- Tests, validation finale et rédaction du rapport.

Cette approche a permis de mener le projet de manière progressive tout en garantissant la qualité des résultats obtenus.

Conclusion

Au cours de ce chapitre, l'organisme d'accueil, le contexte du stage ainsi que les objectifs du projet ont été présentés. La méthodologie adoptée et la planification des différentes phases de réalisation ont également été décrites. Ces éléments constituent le cadre général du projet et permettent d'introduire le chapitre suivant, consacré à l'état de l'art des systèmes de détection d'intrusions et des techniques de Machine Learning.

Chapitre 2 : État de l'art

Introduction

La cybersécurité désigne l'ensemble des moyens techniques, organisationnels et humains déployés pour protéger les systèmes d'information contre les menaces susceptibles de compromettre la confidentialité, l'intégrité et la disponibilité des données. Avec l'explosion des usages numériques et l'interconnexion permanente des équipements, la surface d'attaque n'a cessé de s'étendre, transformant la sécurité informatique en un enjeu stratégique pour les entreprises, les États et les particuliers. La cybersécurité ne se réduit pas à l'installation de pare-feu ou d'antivirus ; elle englobe une démarche globale d'analyse des risques, de prévention, de détection et de réaction face aux incidents.

1. La cybersécurité et les systèmes de détection d'intrusions

1.1 La cybersécurité

La cybersécurité désigne l'ensemble des moyens techniques, organisationnels et humains déployés pour protéger les systèmes d'information contre les menaces susceptibles de compromettre la confidentialité, l'intégrité et la disponibilité des données. Avec l'explosion des usages numériques et l'interconnexion permanente des équipements, la surface d'attaque n'a cessé de s'étendre, transformant la sécurité informatique en un enjeu stratégique pour les entreprises, les États et les particuliers. La cybersécurité ne se réduit pas à l'installation de pare-feu ou d'antivirus ; elle englobe une démarche globale d'analyse des risques, de prévention, de détection et de réaction face aux incidents.

1.2 Les cyberattaques

Une cyberattaque est une action malveillante visant à porter atteinte à un système d'information. Les formes qu'elle peut prendre sont extrêmement variées. Parmi les plus répandues, on distingue :

- **Les dénis de service (DoS/DDoS)** , qui saturent les ressources d'un serveur pour le rendre indisponible ;
- **Les attaques par force brute**, consistant à tester systématiquement des mots de passe pour obtenir un accès non autorisé ;

- **Les intrusions réseau (infiltrations)** , où l'attaquant exploite une vulnérabilité pour pénétrer dans le système ;
- **Les scans de ports**, qui permettent de cartographier les services exposés et de détecter des faiblesses potentielles ;
- **Les attaques web (injection SQL, Cross-Site Scripting)** , ciblant les applications accessibles publiquement.

La sophistication croissante de ces attaques rend indispensable le déploiement de solutions capables de les identifier rapidement et avec précision.

1.3 Les systèmes de détection d'intrusions (IDS)

Un système de détection d'intrusions (Intrusion Detection System, IDS) est un dispositif matériel ou logiciel qui surveille un réseau ou un hôte afin de détecter des activités suspectes ou malveillantes. Sa mission est d'alerter l'administrateur dès qu'une menace potentielle est identifiée, permettant une réaction rapide. L'IDS analyse en continu les événements (paquets réseau, journaux système, appels système) et les confronte à des modèles de comportement connus ou appris. La figure ci-dessous illustre l'architecture générique d'un IDS réseau.

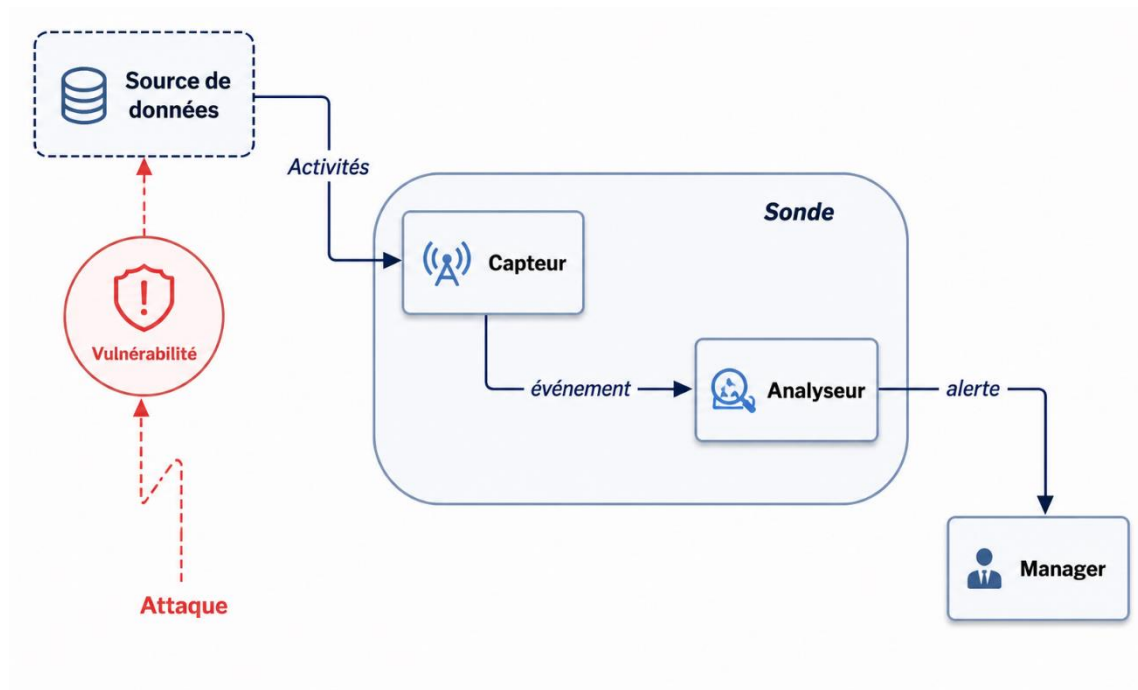


Figure 2: Architecture générique d'un IDS réseau

Le capteur collecte les données brutes, le moteur d'analyse applique des règles ou des modèles de détection, et le module d'alerte notifie l'opérateur en cas d'intrusion suspectée.

1.4 Les types d'IDS

On classe généralement les IDS selon deux critères : la source des données surveillées et la méthode de détection employée.

Selon la source :

- **NIDS (Network-based IDS)** : scrute le trafic réseau au niveau d'un segment stratégique. Il est capable de détecter des attaques visant plusieurs machines, mais peut être aveugle au contenu chiffré.
- **HIDS (Host-based IDS)** : installé sur une machine spécifique, il surveille l'intégrité des fichiers, les appels système et les journaux locaux. Il offre une vision fine des activités sur l'hôte mais ne détecte pas les attaques purement réseau.
- **IDS hybrides** : combinent les deux approches pour une couverture plus large.

Selon la méthode de détection :

- **Détection par signatures** : repose sur une base de signatures d'attaques connues. Très précise pour les menaces répertoriées, elle est inefficace contre les attaques inédites (zero-day).
- **Détection par anomalies** : construit un modèle du comportement normal et signale tout écart significatif. Elle permet de repérer des attaques inconnues mais souffre d'un taux élevé de fausses alertes.

1.5 Les limites des IDS traditionnels

Les IDS fondés uniquement sur des signatures montrent rapidement leurs insuffisances face à l'évolution permanente des menaces. La mise à jour des signatures est un processus lent et réactif, laissant une fenêtre de vulnérabilité. Les approches par anomalies, quant à elles, peinent à définir un profil de normalité stable et génèrent de nombreux faux positifs, ce qui entraîne une lassitude des opérateurs et peut masquer de véritables incidents. De plus, la volumétrie du trafic moderne dépasse souvent les capacités d'analyse en temps réel des systèmes classiques, imposant de recourir à des techniques plus évoluées, capables d'extraire automatiquement des motifs pertinents dans des masses de données hétérogènes. Ces

limitations ont motivé l'intégration de l'apprentissage automatique dans la conception des IDS de nouvelle génération.

2. L'apprentissage automatique appliqué à la détection d'intrusions

2.1 Introduction au Machine Learning

L'apprentissage automatique (Machine Learning, ML) est une branche de l'intelligence artificielle qui consiste à concevoir des algorithmes capables d'apprendre à partir de données, sans être explicitement programmés pour chaque tâche. Plutôt que d'écrire des règles rigides, on fournit au système des exemples à partir desquels il extrait des régularités statistiques, généralisables à de nouvelles situations. Dans le domaine de la cybersécurité, cette capacité à modéliser des comportements complexes à partir d'échantillons de trafic ouvre la voie à des IDS plus adaptatifs et robustes.

2.2 Les types d'apprentissage

On distingue classiquement trois paradigmes :

- **Apprentissage supervisé** : les données d'entraînement sont étiquetées (chaque exemple est associé à une classe, par exemple « Bénin » ou « Attaque »). L'algorithme apprend une fonction de prédiction qui sera utilisée pour classer de nouvelles instances. C'est le cadre naturel pour un IDS disposant d'un jeu de données annoté.
- **Apprentissage non supervisé** : les données ne sont pas étiquetées. L'algorithme cherche des structures cachées (groupes, anomalies). Il peut être utilisé pour la détection d'intrusions sans connaissance préalable des attaques, mais l'interprétation des résultats est plus délicate.
- **Apprentissage semi-supervisé** : combine une petite quantité de données étiquetées avec une grande masse de données non étiquetées, offrant un compromis lorsque l'annotation est coûteuse.

Notre projet s'inscrit exclusivement dans un cadre **supervisé**, car le jeu de données CICIDS2017 fournit des étiquettes fiables et détaillées, permettant un apprentissage discriminant de haute qualité.

2.3 La classification binaire

La classification binaire est un cas particulier de l'apprentissage supervisé où la variable cible ne peut prendre que deux valeurs. Dans le contexte d'un IDS, il s'agit de distinguer le trafic **bénin** du trafic **malveillant**, sans préciser la nature de l'attaque. Cette formulation simplifiée permet d'évaluer la capacité du modèle à détecter une menace, indépendamment de sa catégorie fine. Elle constitue souvent une première étape, facile à interpréter et à mettre en œuvre, et sert de référence pour des tâches plus complexes.

2.4 La classification multi classe

La classification multi classe étend le problème à plus de deux catégories. Appliquée à la détection d'intrusions, elle consiste non seulement à identifier la présence d'une attaque, mais aussi à en préciser le type : DoS, scan de ports, attaque par force brute, infiltration, etc. Cette granularité est précieuse pour l'opérateur, car elle oriente la réponse à apporter. En revanche, elle complexifie la tâche d'apprentissage : le modèle doit apprendre des frontières de décision plus nombreuses, et les classes sont souvent déséquilibrées, certaines attaques étant beaucoup plus rares que d'autres. La résolution de ce défi passe par un choix judicieux d'algorithmes et de métriques d'évaluation adaptées, notamment le Macro F1-Score.

3. Les algorithmes de Machine Learning

Cette section présente les principaux algorithmes d'apprentissage automatique utilisés dans le cadre de la détection d'intrusions. Chacun possède des caractéristiques propres influençant ses performances, sa capacité de généralisation et son coût de calcul. Les algorithmes retenus dans cette étude sont le **Decision Tree**, le **Random Forest**, **XGBoost** et le **Multi-Layer Perceptron (MLP)**.

3.1 Decision Tree

Le **Decision Tree** (arbre de décision) est un algorithme de classification supervisée qui construit un modèle sous forme d'une structure arborescente. À chaque nœud, une caractéristique est sélectionnée afin de diviser les données selon un critère de séparation, jusqu'à obtenir des feuilles représentant les classes finales.

Cet algorithme présente plusieurs avantages. Il est simple à comprendre, facile à interpréter et nécessite peu de préparation des données. De plus, son temps de prédiction est généralement très faible, ce qui le rend adapté aux applications nécessitant une réponse rapide.

Cependant, un arbre de décision peut être sensible au surapprentissage (overfitting), notamment lorsque sa profondeur devient importante. De légères variations dans les données peuvent également conduire à la construction d'arbres différents.

Grâce à sa rapidité d'exécution et à sa simplicité d'interprétation, le Decision Tree constitue une solution intéressante pour les systèmes de détection d'intrusions, en particulier lorsqu'une prise de décision rapide est recherchée.

3.2 Random Forest

Le **Random Forest** est une méthode d'ensemble (Ensemble Learning) qui combine plusieurs arbres de décision afin d'améliorer les performances de classification. Chaque arbre est construit à partir d'un sous-ensemble aléatoire des données et des variables, puis la prédiction finale est obtenue par un vote majoritaire.

Cette approche permet de réduire le risque de surapprentissage tout en améliorant la robustesse et la précision du modèle. Le Random Forest est également capable de gérer des jeux de données volumineux et comportant un grand nombre de variables.

En contrepartie, ce modèle nécessite davantage de ressources de calcul que le Decision Tree, notamment lors de la phase d'entraînement. Son interprétation est également plus complexe puisqu'elle repose sur un ensemble de nombreux arbres.

Grâce à son excellent compromis entre précision, stabilité et capacité de généralisation, le Random Forest est largement utilisé dans les applications de cybersécurité et de détection d'intrusions.

3.3 XGBoost

XGBoost (Extreme Gradient Boosting) est un algorithme d'apprentissage supervisé basé sur la technique du **Gradient Boosting**. Contrairement aux méthodes utilisant plusieurs modèles indépendants, XGBoost construit progressivement une succession d'arbres de décision, chaque nouvel arbre ayant pour objectif de corriger les erreurs commises par les arbres précédents.

Cet algorithme est reconnu pour ses excellentes performances sur de nombreux problèmes de classification et sa capacité à gérer efficacement des jeux de données complexes. Il intègre également plusieurs mécanismes de régularisation permettant de limiter le surapprentissage.

En revanche, XGBoost présente un temps d'entraînement plus important que les arbres de décision classiques et nécessite un réglage plus précis de ses hyperparamètres afin d'obtenir des performances optimales.

Grâce à sa grande précision et à sa robustesse, XGBoost est aujourd'hui considéré comme l'un des algorithmes les plus performants pour les tâches de classification, notamment dans le domaine de la détection d'intrusions.

3.4 Multi-Layer Perceptron (MLP)

Le **Multi-Layer Perceptron (MLP)** est un réseau de neurones artificiels composé d'une couche d'entrée, d'une ou plusieurs couches cachées et d'une couche de sortie. Les informations sont propagées de manière séquentielle entre les différentes couches, tandis que les poids des connexions sont ajustés lors de l'apprentissage grâce à l'algorithme de rétropropagation des erreurs (Backpropagation).

Le principal avantage du MLP réside dans sa capacité à modéliser des relations complexes et non linéaires entre les variables, ce qui le rend particulièrement adapté aux problèmes de classification complexes.

Toutefois, son entraînement est généralement plus coûteux en temps de calcul et nécessite davantage de données ainsi qu'un réglage approprié des hyperparamètres. De plus, son fonctionnement est moins interprétable que celui des modèles basés sur les arbres de décision.

Grâce à sa capacité d'apprentissage des relations complexes présentes dans le trafic réseau, le MLP constitue une approche pertinente pour la détection et la classification des intrusions dans les systèmes informatiques.

4. Jeu de données et évaluation des modèles

4.1 Présentation du jeu de données CICIDS2017

Le jeu de données **CICIDS2017**, élaboré par l'Institut Canadien de Cybersécurité (CIC), est devenu une référence pour l'évaluation des IDS modernes. Capturé sur un environnement réseau réaliste comprenant des machines clientes, un serveur et un pare-feu, il couvre cinq jours de trafic, incluant une activité de fond normale et l'exécution de plusieurs familles d'attaques. Au total, il contient plus de 2,8 millions d'enregistrements décrits par 80 caractéristiques extraites des flux réseau (timing, volumes, flags, etc.). Chaque flux est étiqueté

comme **BENIGN** ou selon une attaque spécifique : DoS Hulk, PortScan, DDoS, FTP-Patator, SSH-Patator, Bot, Web Attack, Infiltration, etc. Cette diversité et cette richesse en font un banc d'essai exigeant mais représentatif des défis réels d'un IDS.

4.2 Prétraitement des données

Avant toute modélisation, le jeu de données doit être rigoureusement nettoyé et préparé. Les étapes classiques comprennent :

- **Suppression des doublons et des valeurs manquantes** : les doublons peuvent biaiser l'apprentissage en accordant un poids artificiel à certains échantillons ; les valeurs manquantes, traitées par suppression ou imputation, garantissent l'intégrité des calculs.
- **Gestion des valeurs aberrantes et infinies** : des caractéristiques telles que des débits calculés peuvent diverger et doivent être bornées ou écrêtées.
- **Normalisation / standardisation** : les caractéristiques présentent des ordres de grandeur très différents (de la fraction de seconde à la centaine de milliers d'octets). Une mise à l'échelle (MinMax, Z-score) est indispensable pour les algorithmes sensibles aux amplitudes, comme le MLP.
- **Encodage des variables catégorielles** : bien que la majorité des attributs du CICIDS2017 soient numériques, d'éventuelles colonnes protocolaires peuvent être transformées via un one-hot encoding.
- **Partition en ensembles d'entraînement et de test** : une séparation temporelle ou aléatoire permet d'évaluer la généralisation réelle du modèle.

4.3 Métriques d'évaluation

La qualité d'un IDS ne se mesure pas uniquement par son taux de bonnes classifications. Dans un contexte où les classes sont fortement déséquilibrées, il est impératif d'utiliser des métriques robustes.

- **Accuracy** : proportion de prédictions correctes. Elle peut être trompeuse si une classe est majoritaire, un modèle prédisant toujours la classe dominante afficherait une accuracy élevée sans jamais détecter les attaques.
- **Precision** : pour une classe donnée, proportion de prédictions positives qui sont effectivement correctes. Une précision élevée limite les fausses alertes.

- **Recall** (sensibilité) : proportion de cas positifs réels correctement identifiés. Un recall élevé signifie que peu d'attaques sont manquées.
- **F1-Score** : moyenne harmonique de la précision et du recall. Il constitue un compromis entre les deux, particulièrement utile quand les classes sont déséquilibrées.
- **Macro F1-Score** : moyenne des F1-Score calculés pour chaque classe, sans pondération par le nombre d'échantillons. Il reflète la capacité du modèle à bien performer sur toutes les classes, y compris les plus rares, ce qui est exactement l'objectif visé dans une classification multi classe d'attaques.

Notre évaluation s'appuiera principalement sur le Macro F1-Score pour sélectionner et comparer les modèles, complété par l'analyse détaillée des matrices de confusion.

Critère	Decision Tree	Random Forest	XGBoost	MLP
Type d'apprentissage	Supervisé	Supervisé	Supervisé	Supervisé
Interprétabilité	Très élevée	Moyenne	Faible	Très faible
Précision	Bonne	Très bonne	Excellente	Très bonne
Résistance au surapprentissage	Faible	Élevée	Élevée	Moyenne
Temps d'entraînement	Faible	Moyen	Élevé	Élevé
Temps de prédiction	Très faible	Faible	Moyen	Faible
Réglage des hyperparamètres	Simple	Moyen	Complexe	Complexe

Le tableau précédent met en évidence les principales caractéristiques des algorithmes étudiés. Le **Decision Tree** se distingue par sa simplicité et sa rapidité d'exécution, tandis que le **Random Forest** améliore la robustesse et la capacité de généralisation grâce à une approche par ensemble. **XGBoost** offre généralement d'excellentes performances au prix d'un entraînement plus coûteux et d'un réglage plus complexe. Enfin, le **MLP** est particulièrement adapté à l'apprentissage de relations non linéaires complexes, mais nécessite davantage de ressources de calcul et une phase d'entraînement plus importante.

4.4 Validation croisée (K-Fold Cross Validation)

La **validation croisée** (*K-Fold Cross Validation*) est une méthode utilisée pour évaluer la capacité de généralisation d'un modèle de Machine Learning. Elle consiste à diviser le jeu de données en **K sous-ensembles** (*folds*) de taille approximativement égale. À chaque itération, le modèle est entraîné sur **K-1 folds** et évalué sur le fold restant. Ce processus est répété **K fois**, de manière à ce que chaque fold soit utilisé une seule fois pour la validation.

Cette approche permet d'obtenir une estimation plus fiable des performances du modèle en exploitant l'ensemble des données disponibles pour l'entraînement et la validation. Elle réduit également l'influence d'un découpage aléatoire des données sur les résultats obtenus.

La figure suivante illustre le principe de fonctionnement de la validation croisée **K-Fold**.

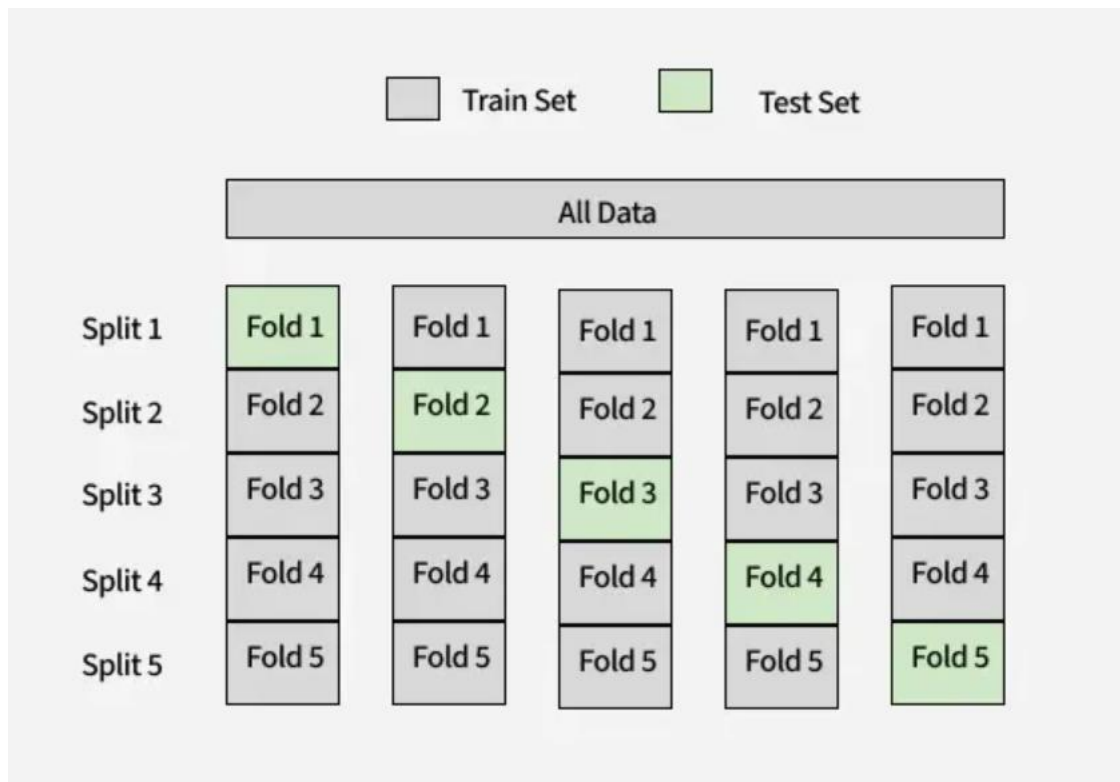


Figure 3: Principe de fonctionnement de la validation croisée K-Fold ($K = 5$)

Dans le cadre de ce projet, une **validation croisée à 5 plis (5-Fold Cross Validation)** a été appliquée afin de vérifier la stabilité et la capacité de généralisation des modèles étudiés.

Conclusion

Ce chapitre a posé les fondements théoriques nécessaires au projet. Les limites des IDS traditionnels ont justifié le recours à l'apprentissage automatique supervisé, aussi bien pour la

classification binaire que multi classe. Les algorithmes retenus – arbre de décision, forêt aléatoire, XGBoost et perceptron multicouche – ont été présentés avec leurs atouts respectifs. Enfin, la description du jeu de données CICIDS2017 et des métriques d'évaluation, en particulier le Macro F1 et la validation croisée, a fixé le cadre méthodologique rigoureux de l'étude expérimentale. Ces bases permettent d'aborder, dans le chapitre suivant, la conception et la modélisation du système de détection d'intrusions.

Chapitre 3 : Conception et modélisation

Introduction

Après avoir présenté les notions théoriques relatives aux systèmes de détection d'intrusions et aux techniques de Machine Learning dans le chapitre précédent, ce chapitre est consacré à la conception de la solution proposée. Il décrit les besoins identifiés, l'architecture générale du système ainsi que les principaux composants qui assurent son fonctionnement. Cette phase de conception constitue une étape essentielle avant la réalisation et le déploiement de l'application.

1. Analyse des besoins

L'analyse des besoins permet d'identifier les fonctionnalités attendues du système ainsi que les contraintes techniques et qualitatives auxquelles il doit répondre. Les besoins sont répartis en deux catégories : les besoins fonctionnels et les besoins non fonctionnels.

1.1 Besoins fonctionnels

Le système proposé doit permettre de :

- Charger un modèle de Machine Learning préalablement entraîné.
- Importer un fichier CSV contenant les caractéristiques du trafic réseau.
- Vérifier la validité des données importées avant la phase de prédiction.
- Prétraiter automatiquement les données selon le pipeline utilisé lors de l'entraînement.
- Détecter automatiquement les intrusions réseau.
- Identifier le type d'attaque détectée (classification multi classe).
- Afficher les résultats des prédictions de manière claire et intuitive.
- Présenter des statistiques et des visualisations facilitant l'interprétation des résultats.
- Permettre le téléchargement des résultats de prédiction.

1.2 Besoins non fonctionnels

Le système doit également satisfaire plusieurs exigences de qualité :

- Offrir une interface utilisateur intuitive et ergonomique.
- Garantir un temps de réponse rapide lors des prédictions.
- Assurer une bonne fiabilité des résultats.

- Être facilement maintenable et évolutif.
- Garantir la sécurité des données manipulées.
- Être compatible avec les principaux navigateurs web.

2. Architecture générale du système

2.1 Architecture proposée

L'architecture retenue repose sur une approche modulaire permettant de séparer les différentes étapes du traitement des données. Le système reçoit les données du trafic réseau via une interface développée avec **Streamlit**. Les données sont ensuite prétraitées avant d'être transmises au modèle de Machine Learning entraîné. Enfin, les résultats de la prédiction sont affichés à l'utilisateur sous forme de tableaux, de graphiques et d'indicateurs de performance.

Le schéma ci-dessous illustre cette architecture modulaire et le flux séquentiel des données, depuis leur capture jusqu'à la visualisation des résultats.

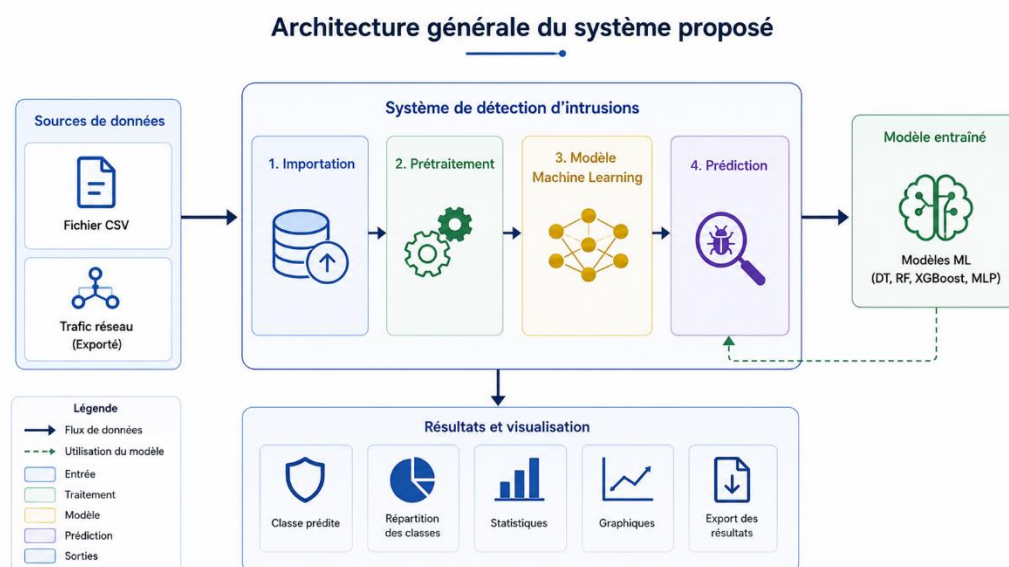


Figure 4: Architecture générale du système de détection d'intrusions

2.2 Description des composants

L'architecture est composée des principaux éléments suivants :

- **Interface utilisateur (Streamlit)** : permet à l'utilisateur d'importer des données, de lancer les prédictions et de consulter les résultats.

- **Module de prétraitement** : applique les mêmes transformations que celles utilisées lors de l'entraînement (sélection des variables, normalisation et préparation des données).
- **Modèle de Machine Learning** : réalise la prédiction des données d'entrée à l'aide du modèle sélectionné (Decision Tree).
- **Module de visualisation** : présente les résultats sous forme de tableaux, de graphiques et d'indicateurs afin de faciliter leur interprétation.
- **Jeu de données** : les données utilisées pour l'entraînement proviennent du jeu de données public **CICIDS2017**, tandis que les données fournies par l'utilisateur sont utilisées uniquement pour la prédiction.

3. Conception de la solution de Machine Learning

Cette section présente la conception de la solution de Machine Learning développée dans le cadre de ce projet. Elle décrit les différentes étapes du pipeline de traitement des données, depuis leur préparation jusqu'à l'entraînement des modèles de classification. Cette conception vise à garantir une détection fiable des intrusions tout en assurant la reproductibilité du processus d'apprentissage.

3.1 Pipelines de traitement des données

Le pipeline de traitement regroupe l'ensemble des opérations appliquées aux données avant leur exploitation par les modèles de Machine Learning. Il permet d'automatiser les différentes étapes de préparation afin d'obtenir des données cohérentes et adaptées à l'apprentissage.

Le pipeline adopté dans ce projet comprend les étapes suivantes :

- Chargement du jeu de données CICIDS2017
- Nettoyage et préparation des données
- Sélection des caractéristiques pertinentes
- Encodage des classes
- Normalisation des variables numériques
- Séparation des données en ensembles d'entraînement et de test
- Entraînement des modèles

- Évaluation des performances.

La figure suivante présente le pipeline complet utilisé dans ce projet.

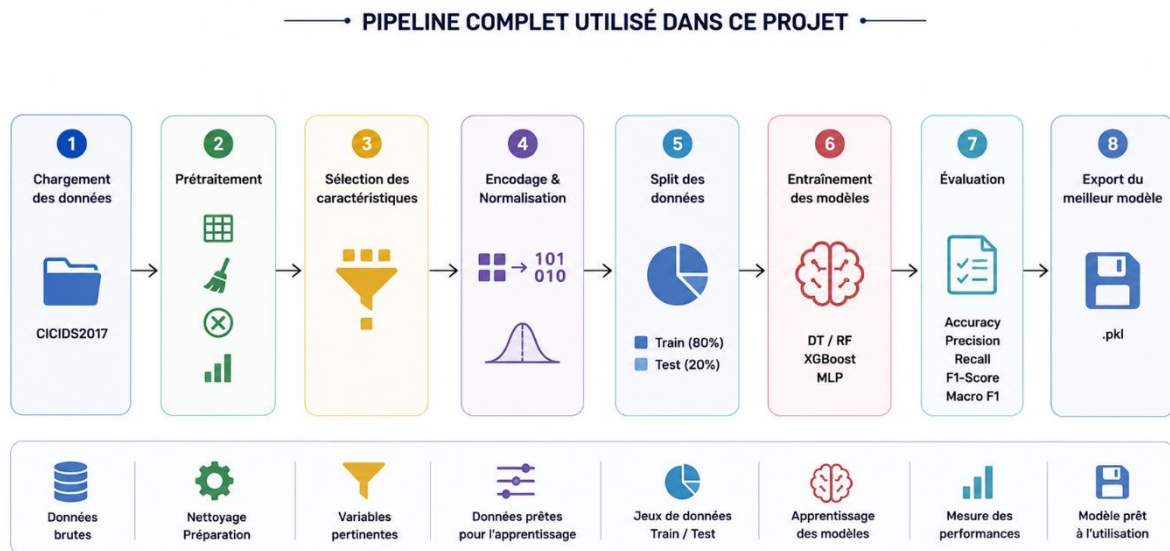


Figure 5: Pipeline complet du projet

3.2 Prétraitement des données

Le prétraitement constitue une étape essentielle dans tout projet de Machine Learning. Il permet d'améliorer la qualité des données avant leur exploitation par les algorithmes d'apprentissage.

Dans ce projet, plusieurs opérations ont été réalisées :

- Fusion des différents fichiers du jeu de données CICIDS2017
- Suppression des valeurs manquantes et infinies
- Élimination des doublons
- Suppression des colonnes constantes ou non pertinentes
- Encodage des étiquettes d'attaque
- Normalisation des variables numériques afin d'harmoniser leurs échelles

Ces traitements permettent de fournir aux modèles des données propres et homogènes, favorisant ainsi un apprentissage plus efficace.

3.3 Sélection des caractéristiques

3.4 Processus d'entraînement des modèles

Après la préparation des données, plusieurs modèles de Machine Learning ont été entraînés afin d'identifier la solution la plus performante.

Les modèles étudiés sont :

- Decision Tree
- Random Forest
- XGBoost
- Multi-Layer Perceptron (MLP)

Les données ont été divisées en un ensemble d'entraînement et un ensemble de test. Les modèles ont ensuite été évalués à l'aide de plusieurs métriques telles que l'Accuracy, la Precision, le Recall, le F1-Score et le Macro F1. Une validation croisée (5-Fold Cross Validation) a également été réalisée afin d'évaluer leur capacité de généralisation.

4. Conception de l'application web

Afin de faciliter l'utilisation du modèle de détection d'intrusions, une application web a été développée avec le framework **Streamlit**. Cette application permet d'effectuer des prédictions de manière simple et interactive tout en offrant une visualisation claire des résultats.

4.1 Architecture de l'application Streamlit

L'application adopte une architecture modulaire composée de plusieurs pages indépendantes assurant chacune une fonctionnalité spécifique.

Elle comprend notamment :

- Une page d'accueil présentant le projet
- Une page de prédiction permettant l'analyse du trafic réseau
- Un tableau de bord affichant les statistiques et les résultats
- Une page de présentation du projet et des technologies utilisées

La figure suivante illustre l'architecture générale de l'application.

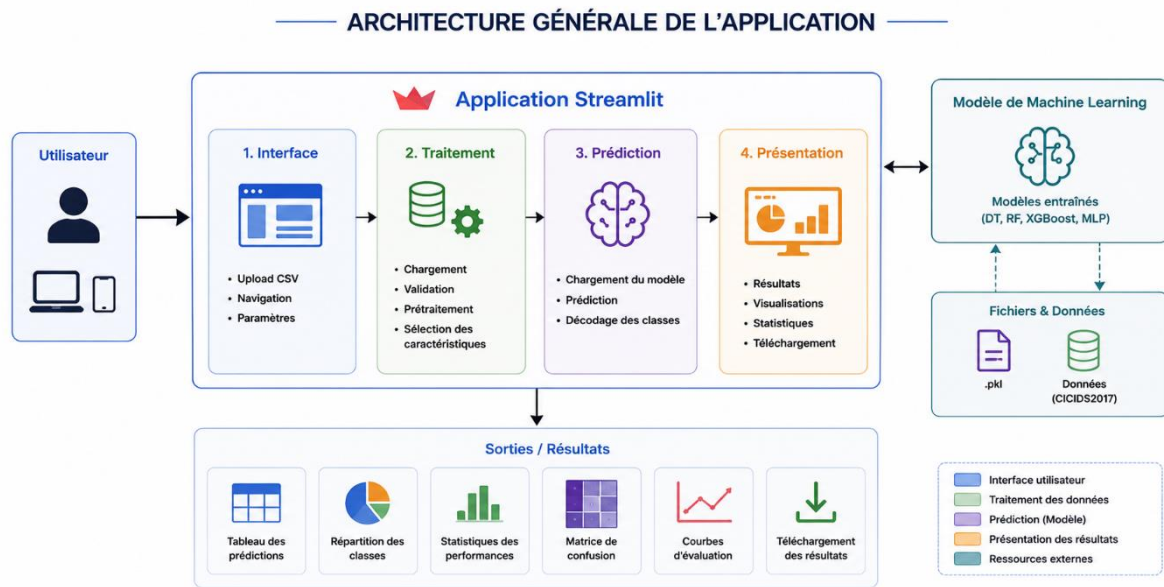


Figure 6: Architecture générale de l'application

4.2 Description des principales interfaces

L'interface utilisateur a été conçue pour offrir une utilisation simple et intuitive.

Les principales interfaces sont les suivantes :

- **Page d'accueil** : présentation du projet et de ses objectifs.
- **Page de prédiction** : importation d'un fichier CSV et lancement des prédictions.
- **Tableau de bord** : affichage des statistiques, graphiques et résultats des prédictions.
- **Page À propos** : présentation du projet, des technologies utilisées et des informations générales.

Chaque interface a été développée dans le but d'améliorer l'expérience utilisateur tout en facilitant l'exploitation des modèles de Machine Learning.

Conclusion

Ce chapitre a présenté la conception de la solution proposée, depuis le pipeline de traitement des données jusqu'à l'architecture de l'application web. Les différentes composantes du système ont été décrites afin de mettre en évidence leur rôle dans le processus de détection des intrusions. Le chapitre suivant est consacré à la réalisation du système ainsi qu'à l'analyse des résultats expérimentaux obtenus.

Chapitre 4 : Expérimentation et résultats

Introduction

Après la phase de conception, la solution proposée a été mise en œuvre et évaluée à l'aide de plusieurs modèles de Machine Learning. Ce chapitre présente tout d'abord l'environnement de développement utilisé, puis les différentes expérimentations réalisées ainsi que l'analyse des résultats obtenus. L'objectif est de comparer les performances des modèles étudiés et d'identifier la solution la plus adaptée à la détection d'intrusions réseau.

1. Environnement de développement

La réalisation de ce projet s'est appuyée sur un ensemble de technologies et d'outils permettant le développement, l'entraînement, l'évaluation et le déploiement des modèles de Machine Learning. L'environnement technique a été choisi afin d'assurer une bonne compatibilité entre les différentes étapes du projet, depuis le prétraitement des données jusqu'à la mise en œuvre de l'application web.

1.1 Langage de programmation

Le projet a été entièrement développé en **Python**, un langage largement utilisé dans le domaine de la science des données et de l'intelligence artificielle grâce à son vaste écosystème de bibliothèques dédiées au Machine Learning.



Figure 7: Python Logo

1.2 Bibliothèques et frameworks

Les principales bibliothèques utilisées sont :

Bibliothèque	Rôle
Pandas	Manipulation et analyse des données
NumPy	Calcul scientifique

Scikit-learn	Prétraitement, entraînement et évaluation des modèles
XGBoost	Implémentation du modèle XGBoost
TensorFlow / Keras	Développement du modèle MLP
Matplotlib	Création de graphiques
Seaborn	Visualisation statistique
Joblib	Sauvegarde et chargement des modèles
Streamlit	Développement de l'application web

1.3 Outils de développement

Les outils suivants ont été utilisés au cours du projet :

Outil	Utilisation
Visual Studio Code	Développement de l'application
Jupyter Notebook	Analyse des données et expérimentation
Git	Gestion des versions
GitHub	Hébergement du code source et collaboration
Streamlit	Déploiement local de l'application web

1.4 Jeu de données utilisée

Le jeu de données exploité est **CICIDS2017** dans sa version complète, téléchargée depuis le site officiel de l'Institut Canadien de Cybersécurité. Il regroupe les captures de cinq jours et comprend les fichiers CSV correspondant au trafic bénin et aux différentes attaques. Une description plus détaillée est fournie ci-après.

2. Réalisation du système

2.1 Présentation du jeu de données

Les expérimentations réalisées dans le cadre de ce projet reposent sur le jeu de données public **CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017)**. Ce jeu de données est l'un des plus utilisés dans la recherche en cybersécurité pour l'évaluation des systèmes de détection d'intrusions basés sur le Machine Learning.

CICIDS2017 reproduit un environnement réseau réaliste comprenant à la fois du trafic normal (*BENIGN*) et plusieurs catégories d'attaques représentatives des menaces actuelles. Les flux réseau ont été capturés pendant cinq jours de trafic, puis transformés en fichiers CSV contenant des caractéristiques statistiques décrivant chaque connexion réseau.

Le jeu de données est réparti en huit fichiers correspondant aux différents jours et périodes de capture. Chaque fichier contient **79 colonnes**, dont **78 caractéristiques** décrivant les flux réseau ainsi qu'une colonne représentant la classe (*Label*). Les différentes attaques présentes incluent notamment les attaques **DoS**, **DDoS**, **PortScan**, **Bot**, **FTP-Patator**, **SSH-Patator**, **Infiltration**, ainsi que plusieurs attaques Web.

Le tableau suivant présente les principales caractéristiques des différents fichiers constituant le jeu de données.

Fichier	Nombre d'instances	Nombre de variables	Valeurs manquantes	Valeurs infinies	Doublons	Classes
Friday-WorkingHours-Afternoon-DDos	225 745	79	4	64	2 633	DDoS, BENIGN
Friday-WorkingHours-Afternoon-PortScan	286 467	79	15	727	72 353	PortScan, BENIGN

Friday- WorkingHours -Morning	191 033	79	28	216	6 888	BENIGN, Bot
Monday- WorkingHours	529 918	79	64	810	26 935	BENIGN
Thursday- WorkingHours -Afternoon- Infiltration	288 602	79	18	396	35 630	BENIGN, Infiltration
Thursday- WorkingHours -Morning- WebAttacks	170 366	79	20	250	6 066	BENIGN, Web Attacks
Tuesday- WorkingHours	445 909	79	201	327	24 065	BENIGN, FTP- Patator, SSH- Patator
Wednesday- WorkingHours	692 703	79	1 008	1 586	81 909	BENIGN, DoS, Heartblee d

Afin de fournir une vue d'ensemble du jeu de données utilisé, le tableau suivant résume ses principales caractéristiques.

Caractéristique	Valeur
Nombre total de fichiers	8
Nombre total d'instances	2 830 743

Nombre de caractéristiques	78
Nombre total de colonnes	79
Nombre de classes	15
Classe majoritaire	BENIGN
Jeu de données	CICIDS2017

L'analyse préliminaire montre que certains fichiers contiennent des **valeurs manquantes**, des **valeurs infinies** ainsi qu'un nombre important de **doublons**. Ces anomalies nécessitent une phase de prétraitement afin d'améliorer la qualité des données avant l'entraînement des modèles de Machine Learning. Les étapes de nettoyage et de préparation des données sont présentées dans la section suivante.

2.2 Analyse exploratoire des données (EDA)

Une analyse exploratoire des données (*Exploratory Data Analysis - EDA*) a été réalisée afin de mieux comprendre la structure du jeu de données **CICIDS2017** avant son prétraitement. Cette étape a permis d'examiner les huit fichiers constituant le jeu de données, d'identifier les différentes catégories d'attaques présentes et d'évaluer la qualité des données.

L'analyse a également mis en évidence plusieurs caractéristiques importantes, notamment la présence de valeurs manquantes, de valeurs infinies et de doublons dans certains fichiers. Par ailleurs, l'étude de la distribution des classes a révélé un important déséquilibre entre le trafic normal (**BENIGN**) et les différentes catégories d'attaques, ce qui constitue une difficulté classique dans les systèmes de détection d'intrusions.

La figure suivante présente la répartition des classes dans les différents fichiers du jeu de données, illustrant la diversité des attaques ainsi que la prédominance du trafic normal.

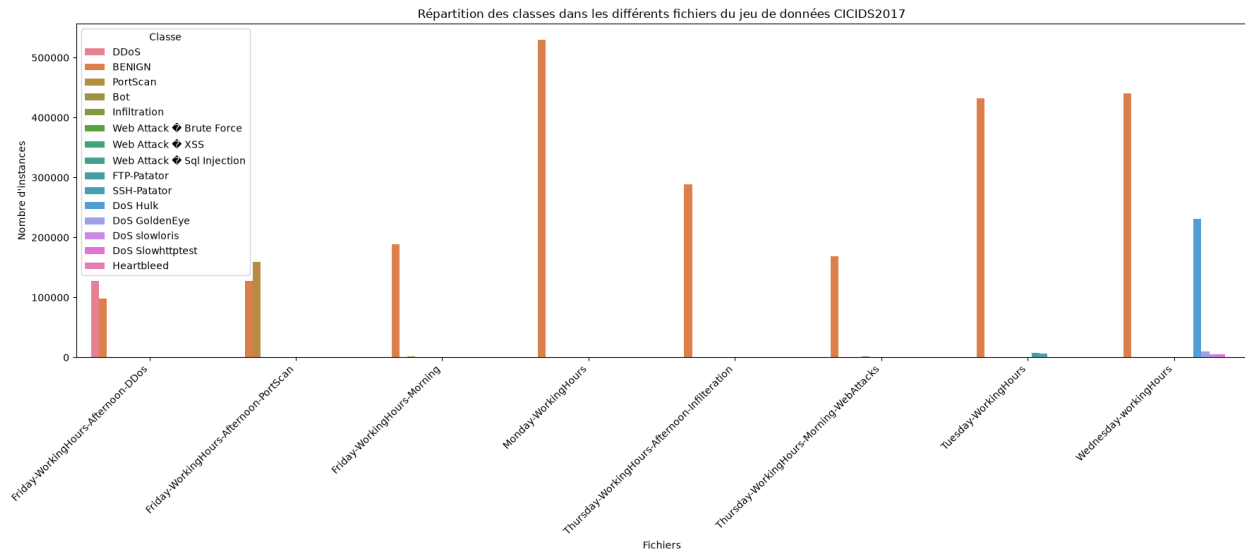


Figure 8: Répartition des classes dans les différents fichiers du jeu de données

Les observations issues de cette analyse ont orienté les étapes de prétraitement décrites dans la section suivante, notamment le nettoyage des données et la préparation des variables destinées à l'entraînement des modèles de Machine Learning.

2.3 Prétraitement des données

Le prétraitement des données constitue une étape essentielle dans tout projet de Machine Learning. La qualité des données utilisées influence directement les performances et la capacité de généralisation des modèles. Avant l'entraînement, le jeu de données **CICIDS2017** a donc fait l'objet d'un ensemble d'opérations visant à produire un jeu de données propre, cohérent et exploitable.

2.3.1 Fusion des fichiers

Le jeu de données CICIDS2017 est initialement réparti en **huit fichiers CSV**, chacun correspondant à une journée ou une période de capture du trafic réseau. Afin de simplifier le traitement des données et de permettre l'entraînement des modèles sur un ensemble unique, ces huit fichiers ont été fusionnés en un seul jeu de données.

Cette opération a permis de regrouper l'ensemble des flux réseau, qu'ils soient légitimes ou malveillants, dans une base de données unique facilitant les étapes de prétraitement et d'apprentissage.

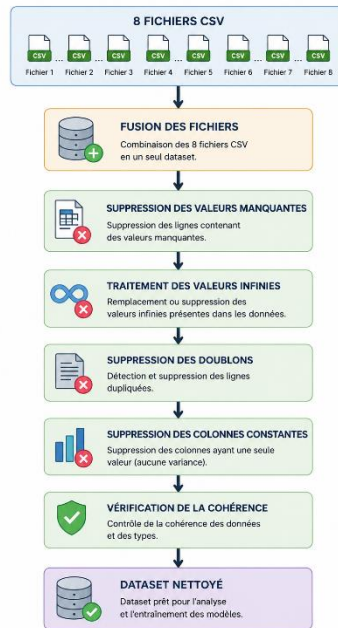


Figure 9: Pipeline de nettoyage des données

2.3.2 Nettoyage des données

Après la fusion, plusieurs opérations de nettoyage ont été réalisées afin d'améliorer la qualité du jeu de données.

Les principales étapes sont les suivantes :

- Suppression des valeurs manquantes (*Missing Values*) ;
- Remplacement puis suppression des valeurs infinies (*Infinite Values*) ;
- Suppression des lignes dupliquées (*Duplicates*) ;
- Suppression des colonnes constantes ne contenant aucune information discriminante ;
- Vérification de la cohérence des données.

Ces traitements permettent d'éliminer les informations inutiles ou erronées susceptibles d'affecter les performances des modèles de Machine Learning.

Le tableau suivant présente l'état du jeu de données avant et après le prétraitement.

Indicateur	Avant prétraitement	Après prétraitement
Nombre d'instances	2 830 743	2 520 798
Nombre de colonnes	79	79

Valeurs manquantes	Présentes	0
Valeurs infinies	Présentes	0
Doublons	Présents	0
Colonnes constantes	Présentes	Supprimées

À l'issue de cette phase, un nouveau jeu de données propre a été obtenu et sauvegardé afin d'être réutilisé dans les différentes expérimentations sans avoir à répéter les opérations de nettoyage. Ce jeu de données constitue la base utilisée pour toutes les étapes d'entraînement, de validation et d'évaluation des modèles de Machine Learning présentées dans la suite de ce chapitre.

2.4 Classification binaire

Une première série d'expérimentations a été réalisée en **classification binaire**, dont l'objectif est de distinguer le trafic réseau normal (*BENIGN*) du trafic malveillant (*ATTACK*). Pour cela, toutes les catégories d'attaques du jeu de données CICIDS2017 ont été regroupées en une seule classe « Attack ».

Afin d'étudier l'influence de certaines caractéristiques sur les performances des modèles, deux configurations ont été considérées :

- Une première configuration utilisant l'ensemble des caractéristiques, y compris **Destination Port** ;
- Une seconde configuration dans laquelle la caractéristique **Destination Port** a été supprimée avant l'entraînement.

Ces deux scénarios permettent d'évaluer la dépendance des modèles vis-à-vis de cette caractéristique et de mesurer leur capacité de généralisation.

Les expérimentations ont été réalisées à l'aide des trois modèles suivants :

- Decision Tree ;
- Random Forest ;
- XGBoost.

La figure suivante illustre la répartition des deux classes utilisées pour cette expérimentation.

Figure 4.X : Répartition des classes de la classification binaire (BENIGN / ATTACK).

Les performances obtenues dans les deux configurations (avec et sans *Destination Port*) sont présentées dans la section des résultats expérimentaux.

2.5 Classification multi classe

Après la classification binaire, une seconde expérimentation a été menée en **classification multi classe**. Contrairement au scénario précédent, cette approche consiste à identifier précisément le type d'attaque présent dans chaque flux réseau.

Quatre modèles de Machine Learning ont été entraînés et évalués :

- Decision Tree
- Random Forest
- XGBoost
- Multi-Layer Perceptron (MLP).

Chaque modèle a été entraîné sur le jeu de données prétraité, puis évalué à l'aide des métriques suivantes : Accuracy, Precision, Recall, Weighted F1-score, Macro F1-score, ainsi que les temps d'entraînement et de prédiction.

Les résultats détaillés de ces expérimentations sont présentés dans les sections suivantes.

2.6 Développement de l'application web

Afin de rendre le modèle retenu facilement exploitable, une application web interactive a été développée à l'aide du framework **Streamlit**. Cette application permet de charger des données réseau, d'effectuer une prédiction à partir du modèle entraîné et d'afficher le résultat de manière simple et intuitive.

Le modèle final retenu a été sauvegardé au format **Joblib (.pkl)** puis intégré directement dans l'application afin de réaliser les prédictions en temps réel.

3. Résultats expérimentaux

Cette section présente les performances obtenues par les différents modèles de Machine Learning dans les scénarios de classification binaire et multi classe.

3.1 Résultats de la classification binaire

Cette section présente les résultats obtenus lors des expérimentations de classification binaire. L'objectif est d'évaluer la capacité des modèles de Machine Learning à distinguer le trafic réseau normal (**BENIGN**) du trafic malveillant (**ATTACK**). Deux scénarios ont été étudiés : la première conserve la caractéristique **Destination Port**, tandis que le second est réalisé après sa suppression afin d'évaluer son influence sur les performances des modèles.

3.1.1 Classification binaire avec Destination Port

Dans cette première expérimentation, les modèles **Decision Tree**, **Random Forest** et **XGBoost** ont été entraînés en utilisant l'ensemble des caractéristiques disponibles, y compris la variable **Destination Port**.

Afin d'analyser plus finement la qualité des prédictions, les matrices de confusion des différents modèles sont présentées ci-dessous.

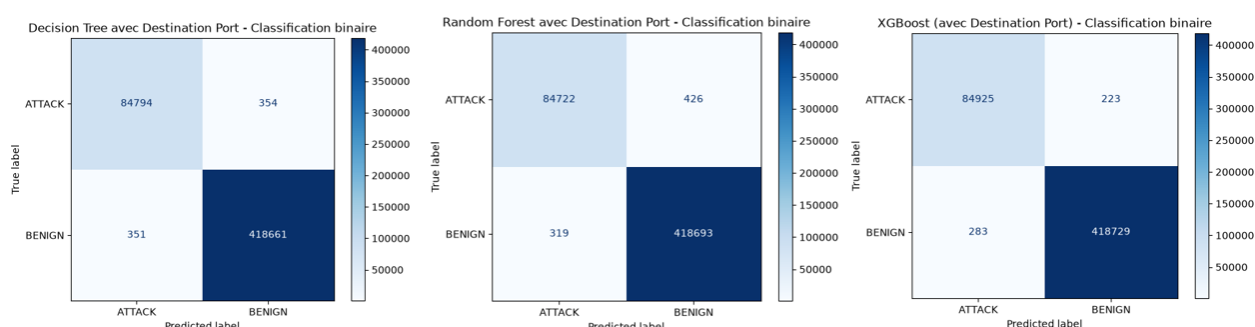


Figure 10: Matrices de confusion des algorithmes de ML avec destination port

Les résultats montrent que les trois modèles obtiennent d'excellentes performances pour la détection du trafic malveillant, avec une très faible proportion d'erreurs de classification.

3.1.2 Classification binaire sans Destination Port

Dans cette seconde expérimentation, la caractéristique **Destination Port** a été supprimée avant l'entraînement des modèles afin d'évaluer son impact sur les performances de détection.

Les matrices de confusion permettent ensuite d'observer les erreurs de classification produites par chaque modèle.

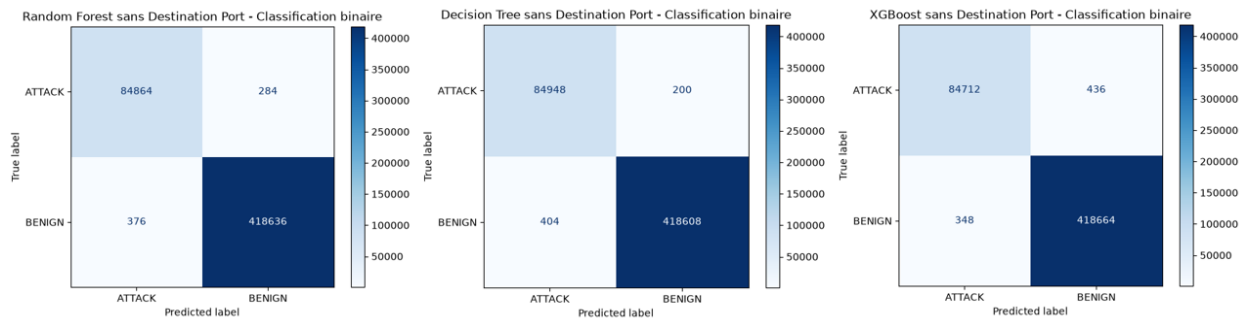


Figure 11: Matrices de confusion des modèles sans Destination Port

Malgré la suppression de cette variable, les modèles conservent des performances élevées, ce qui montre leur capacité à exploiter les autres caractéristiques du trafic réseau.

3.1.3 Comparaison des deux configurations

Afin d'évaluer l'influence de la variable **Destination Port** sur les performances des modèles de Machine Learning, une comparaison a été réalisée entre deux configurations : la première utilisant l'ensemble des caractéristiques disponibles et la seconde après suppression de cette variable.

Les résultats obtenus montrent que la présence de **Destination Port** améliore légèrement les performances globales des trois modèles, notamment en termes d'Accuracy, de Precision et de F1-score. Cette amélioration s'explique par le fait que certains services réseau utilisent des ports spécifiques, ce qui constitue une information pertinente pour différencier certaines catégories de trafic légitime et malveillant.

Toutefois, les performances restent très élevées après la suppression de cette caractéristique. Cette observation montre que les modèles ne reposent pas uniquement sur le numéro de port pour effectuer leurs prédictions, mais exploitent également les nombreuses caractéristiques comportementales du trafic réseau telles que la durée des connexions, les statistiques de paquets, les tailles des flux ou encore les indicateurs temporels.

Ces résultats mettent ainsi en évidence l'importance du **feature engineering** dans la conception d'un système de détection d'intrusions. En sélectionnant des caractéristiques représentatives du comportement réel des communications réseau, il est possible de construire des modèles robustes, capables de maintenir d'excellentes performances même en l'absence d'une variable pourtant considérée comme informative.

Dans le contexte des **Network Intrusion Detection Systems (NIDS)**, cette approche est particulièrement intéressante, car un système qui dépend excessivement d'une seule

caractéristique, telle que **Destination Port**, risque de voir ses performances diminuer face à des attaques utilisant des ports dynamiques, des techniques d'obfuscation ou des protocoles non standards. À l'inverse, un modèle reposant sur un ensemble diversifié de caractéristiques est généralement plus robuste et offre une meilleure capacité de généralisation face à de nouveaux scénarios d'attaque.

3.2 Résultats de la classification multi classe

Après avoir validé l'approche binaire, les modèles ont été évalués sur le problème de **classification multi-class**. Cette expérimentation consiste à identifier précisément la catégorie de chaque attaque présente dans le jeu de données CICIDS2017. Les performances ont été évaluées à l'aide des métriques **Accuracy**, **Precision**, **Recall**, **Weighted F1-score** et **Macro F1-score**, complétées par les rapports de classification et les matrices de confusion.

3.2.1 Résultats du modèle *Decision Tree*

Le modèle **Decision Tree** a obtenu d'excellentes performances globales avec une **accuracy de 99,81 %**, un **Weighted F1-score de 99,81 %** et un **Macro F1-score de 88,91 %**. Les résultats montrent une excellente détection des classes majoritaires, tandis que les performances diminuent légèrement pour certaines attaques très peu représentées, notamment *Heartbleed* et *Web Attack XSS*.



Figure 12: Matrice De Confusion Decision Tree Multi Class

3.2.2 Résultats du modèle Random Forest

Le modèle **Random Forest** présente les meilleures performances globales parmi les modèles étudiés avec une **accuracy de 99,82 %**, un **Weighted F1-score de 99,82 %** et un **Macro F1-score de 88,72 %**. Il améliore légèrement les performances sur plusieurs classes tout en conservant un très faible taux d'erreur.



Figure 13: Matrice De Confusion Random Forest Multi Class

3.2.3 Résultats du modèle XGBoost

Le modèle **XGBoost** obtient une **accuracy de 99,26 %** et un **Weighted F1-score de 99,27 %**. Bien que ses performances restent élevées sur les classes les plus représentées, il rencontre davantage de difficultés pour identifier certaines classes rares, ce qui se traduit par un **Macro F1-score de 63,98 %**, nettement inférieur à celui des modèles basés sur les arbres de décision.

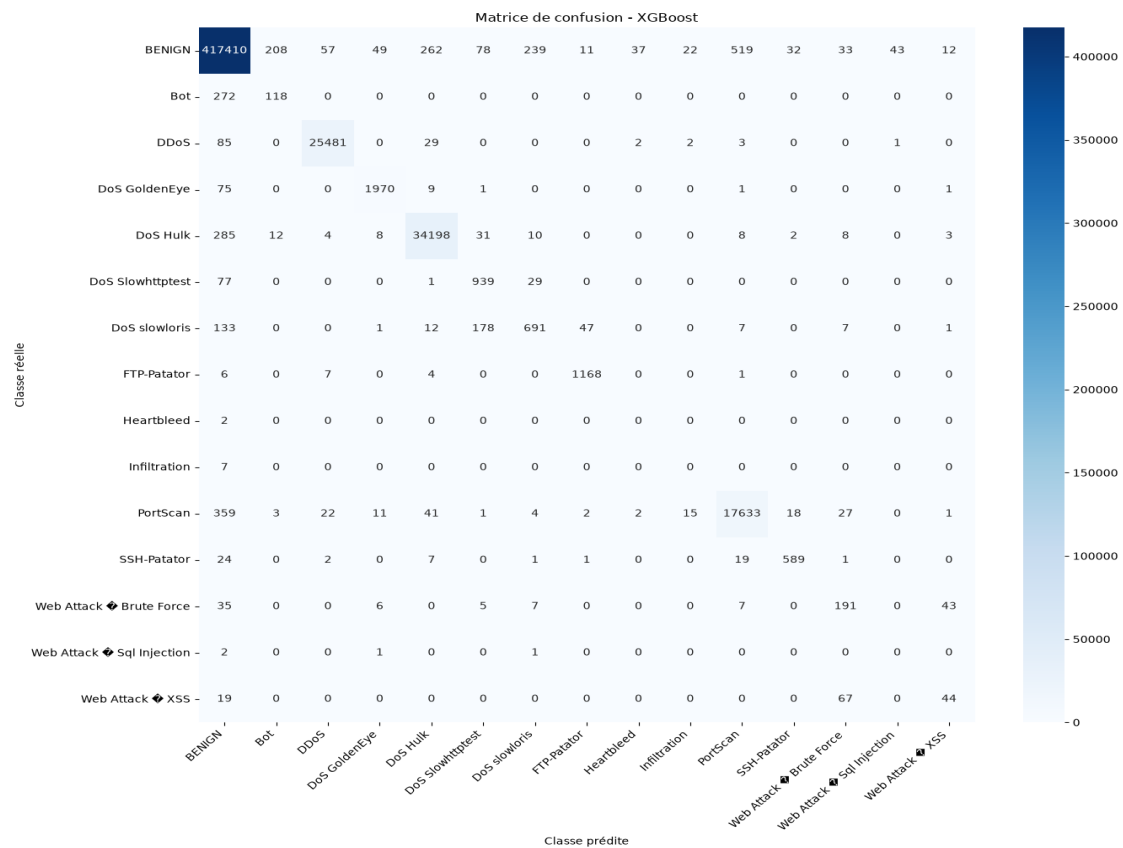


Figure 14: Matrice De Confusion XGBoost Multi Class

3.3 Comparaison des modèles de Machine Learning

Afin d'identifier le modèle le plus performant pour la détection d'intrusions, les trois algorithmes ont été comparés selon les principales métriques d'évaluation ainsi que leurs temps d'entraînement et de prédiction.

Le tableau suivant synthétise les résultats obtenus.

Modèle	Accuracy	Precision	Recall	Weighted F1	Macro F1	Training Time	Temps de prédiction
Decision Tree	0.998064	0.998074	0.998064	0.998068	0.889147	235.12	0.38
Random Forest	0.998229	0.998191	0.998229	0.998201	0.887164	397.12	2.98
XGBoost	0.992606	0.992805	0.992606	0.992686	0.639818	451.38	5.43

Les résultats montrent que **Random Forest** obtient la meilleure accuracy (**99,82 %**) ainsi que le meilleur Weighted F1-score. Cependant, cette légère amélioration s'accompagne d'un temps d'entraînement (**397,12 s**) et d'un temps de prédiction (**2,98 s**) nettement supérieurs à ceux du **Decision Tree**.

Le **Decision Tree** présente des performances quasiment équivalentes tout en étant le modèle le plus rapide, avec un temps d'entraînement de **235,12 s** et un temps de prédiction inférieur à **0,4 seconde**. Son **Macro F1-score (88,91 %)** est également légèrement supérieur à celui de Random Forest, ce qui traduit une meilleure prise en compte des classes minoritaires.

En revanche, **XGBoost** obtient des performances inférieures sur ce jeu de données, notamment pour les attaques peu représentées. Son **Macro F1-score de 63,98 %** ainsi que ses temps d'entraînement et de prédiction plus élevés montrent qu'il est moins adapté à cette problématique dans les conditions expérimentales retenues.

Au regard de ces résultats, le **Decision Tree** a été retenu comme modèle final du système. Il offre le meilleur compromis entre précision, rapidité d'exécution, simplicité de déploiement et capacité de généralisation, ce qui justifie son intégration dans l'application web développée avec Streamlit.

3.4 Validation par K-Fold Cross Validation

Afin d'évaluer la stabilité et la capacité de généralisation du modèle retenu (**Decision Tree**), une validation croisée à **5 plis (5-Fold Cross Validation)** a été réalisée. Cette méthode consiste à diviser le jeu de données d'entraînement en cinq sous-ensembles de taille équivalente. À chaque itération, quatre plis sont utilisés pour l'entraînement tandis que le pli restant est réservé à la validation. L'opération est répétée cinq fois afin que chaque pli soit utilisé une seule fois comme ensemble de validation.

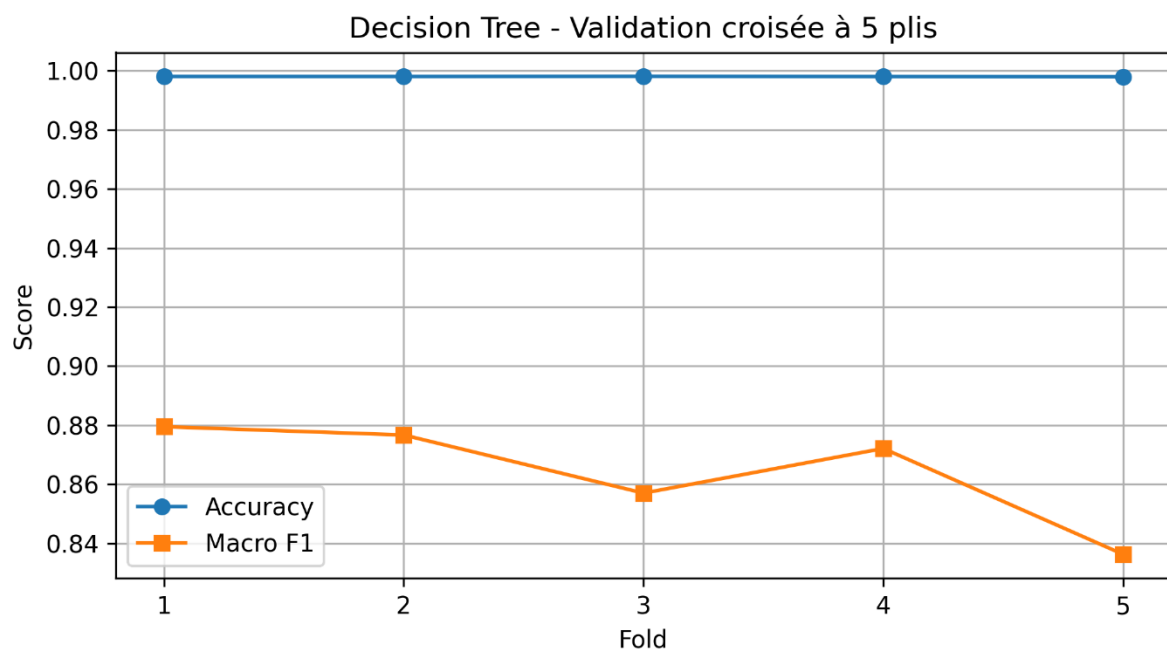
Les performances obtenues pour chacun des cinq plis sont présentées dans le tableau suivant. Les résultats montrent une très faible variation entre les différentes itérations, ce qui traduit une excellente stabilité du modèle.

Fold	Accuracy	Precision	Recall	Weighted F1	Macro F1
1	0.998068	0.998073	0.998068	0.998068	0.879471
2	0.998056	0.998049	0.998056	0.998050	0.876653

3	0.998114	0.998124	0.998114	0.998117	0.856981
4	0.998036	0.998047	0.998036	0.998040	0.872087
5	0.997975	0.997983	0.997975	0.997977	0.836256
Moyenne ±	0.998050 ±	0.998055 ±	0.998050 ±	0.998051 ±	0.864290 ±
Écart-type	0.000045	0.000046	0.000045	0.000045	0.016021

À l'issue des cinq itérations, le modèle obtient une accuracy moyenne de 99,805 % $\pm$ 0,0045 %, une précision moyenne de 99,8055 % $\pm$ 0,0046 %, un recall moyen de 99,8050 % $\pm$ 0,0045 % ainsi qu'un Weighted F1-score moyen de 99,8051 % $\pm$ 0,0045 %. Le Macro F1-score atteint quant à lui une valeur moyenne de 86,43 % $\pm$ 1,60 %.

La Figure ci-dessous illustre l'évolution des performances obtenues sur chacun des cinq plis. On observe que les métriques principales (Accuracy, Precision, Recall et Weighted F1-score) restent quasiment constantes d'un pli à l'autre, tandis que le Macro F1-score présente une légère variabilité, principalement due au déséquilibre entre les différentes classes d'attaques.



Ces résultats confirment la robustesse et la bonne capacité de généralisation du modèle Decision Tree. Les écarts-types très faibles obtenus pour les principales métriques montrent que les performances du modèle ne dépendent pas d'une partition particulière des données, ce qui renforce la fiabilité des résultats obtenus.

3.5 Analyse et interprétation des résultats

L'analyse des résultats montre que les modèles basés sur les arbres de décision offrent les meilleures performances sur le jeu de données CICIDS2017. En classification binaire, les trois modèles atteignent des performances très élevées, tandis que la suppression de la variable **Destination Port** permet d'évaluer leur robustesse vis-à-vis de cette caractéristique.

En classification multi classe, le **Decision Tree** présente un excellent compromis entre précision, rapidité d'exécution et simplicité de déploiement. Les résultats de la validation croisée confirment également la stabilité du modèle, avec des performances très proches sur les cinq plis, ce qui démontre une bonne capacité de généralisation.

Au regard de l'ensemble des expérimentations réalisées, le modèle **Decision Tree** a été retenu pour être intégré à l'application web développée dans le cadre de ce projet.

4. Présentation de l'application

4.1 Page d'accueil

4.2 Tableau de bord

4.3 Interface de prédiction

4.4 Affichage des résultats

Conclusion

Ce chapitre a présenté les différentes expérimentations réalisées ainsi que les résultats obtenus pour la détection d'intrusions à l'aide des modèles de Machine Learning. Après les étapes de prétraitement des données, les modèles ont été évalués en classification binaire et multi classe, puis comparés selon plusieurs métriques de performance. Les résultats de la validation croisée ont confirmé la robustesse du modèle retenu, tandis que l'application web développée avec Streamlit a permis de démontrer son intégration dans un outil fonctionnel de détection d'intrusions.

CONCLUSION GÉNÉRALE

Ce projet de stage avait pour objectif de concevoir et de développer un système de détection d'intrusions basé sur le Machine Learning, capable d'identifier efficacement les activités malveillantes à partir du jeu de données public **CICIDS2017**.

Dans un premier temps, une étude bibliographique a permis de présenter les concepts fondamentaux liés aux systèmes de détection d'intrusions, aux jeux de données publics ainsi qu'aux principales techniques de Machine Learning utilisées dans ce domaine. Cette analyse a servi de base pour la conception de la solution proposée.

Par la suite, un important travail de préparation des données a été réalisé. Les différents fichiers du jeu de données ont été fusionnés, nettoyés et prétraités afin d'obtenir un ensemble de données fiable et exploitable pour l'entraînement des modèles. Une attention particulière a été portée à la qualité des données, notamment par le traitement des valeurs manquantes, des valeurs infinies, des doublons ainsi que par la sélection des caractéristiques les plus pertinentes.

Les expérimentations ont ensuite permis d'évaluer plusieurs modèles de Machine Learning, notamment **Decision Tree**, **Random Forest**, **XGBoost** et **MLP**, dans des scénarios de classification binaire et multiclasse. Les résultats obtenus montrent que les modèles étudiés atteignent d'excellentes performances, avec une accuracy supérieure à 99 % pour les meilleurs algorithmes. La validation croisée (5-Fold Cross Validation) a également confirmé la robustesse et la bonne capacité de généralisation du modèle retenu.

Enfin, le modèle **Decision Tree** a été intégré dans une application web développée avec **Streamlit**, permettant de réaliser des prédictions de manière simple et interactive. Cette application constitue une preuve de concept démontrant qu'un modèle de Machine Learning peut être intégré dans un outil d'aide à la détection des intrusions réseau.

Bien que les résultats obtenus soient très satisfaisants, certaines perspectives d'amélioration peuvent être envisagées. Il serait notamment intéressant d'évaluer le système sur des jeux de données plus récents, de tester des modèles de Deep Learning ou encore de connecter l'application à un système de capture de trafic en temps réel afin de réaliser une détection continue des intrusions.

En conclusion, ce projet a permis de mettre en pratique les connaissances acquises en cybersécurité, en science des données et en Machine Learning, tout en démontrant l'intérêt de ces techniques pour renforcer la détection des menaces dans les réseaux informatiques.

BIBLIOGRAPHIE

- [1] I. Sharafaldin, A. Habibi Lashkari et A. A. Ghorbani, *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*, Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), Funchal, Portugal, 2018.
- [2] Canadian Institute for Cybersecurity (CIC), *CICIDS2017 Dataset*, University of New Brunswick, 2017. Disponible sur : <https://www.unb.ca/cic/datasets/ids-2017.html>
- [3] T. Hastie, R. Tibshirani et J. Friedman, *The Elements of Statistical Learning*, 2^e édition, Springer, 2009.
- [4] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [5] L. Breiman, "Random Forests", *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] J. R. Quinlan, *C4.5: Programs for Machine Learning*, Morgan Kaufmann Publishers, 1993.
- [7] T. Chen et C. Guestrin, "XGBoost: A Scalable Tree Boosting System", *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, USA, 2016.
- [8] S. Haykin, *Neural Networks and Learning Machines*, 3^e édition, Pearson Education, 2009.
- [9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] Scikit-learn Developers, *Scikit-learn Documentation*. Disponible sur : <https://scikit-learn.org/>
- [11] Streamlit Inc., *Streamlit Documentation*. Disponible sur : <https://docs.streamlit.io/>
- [12] Pandas Development Team, *Pandas Documentation*. Disponible sur : <https://pandas.pydata.org/docs/>
- [13] NumPy Developers, *NumPy Documentation*. Disponible sur : <https://numpy.org/doc/>
- [14] Matplotlib Development Team, *Matplotlib Documentation*. Disponible sur : <https://matplotlib.org/>
- [15] Jupyter Project, *Project Jupyter Documentation*. Disponible sur : <https://jupyter.org/>